LA2 PROJECT REPORT

Multi-Horizon Surface-Water Nitrate Forecasting

Using NEON 15-minute measurements aggregated to 30-minute intervals

Student package	Person3
Assigned NEON sites	LECO, LEWI, MCDI
Data product	Nitrate in Surface Water - DP1.20033.001
Original measurement	surfWaterNitrateMean (micromoles per liter, µmol/L)
Forecast horizons	24 hours ahead and 7 days ahead
Implementation	PySpark data processing + person-specific Python estimator libraries; notebook and source PDF included in this package

Objective. Use historical nitrate measurements, time-based features and recent nitrate history to estimate future nitrate means at the same monitoring sites. Assess two useful forecast horizons, compare statistical/ML regressors against naive baselines, and measure computation time and sample-size scaling.

Package contents: report PDF; complete PySpark notebook (.ipynb); printed PySpark source PDF and .py source; original site-specific NEON measurement CSVs; source manifest; data dictionary; preprocessing notes; reference result tables and figures.

1. Problem Statement and Research Questions

NEON provides nitrate concentration summaries at 15-minute intervals. The research task is to determine how well historical nitrate measurements and time features can forecast the nitrate mean 24 hours and 7 days into the future for the three assigned sites. This is a supervised regression problem with a temporal rather than random train/test split.

Research questions: (1) Which of K-Nearest Neighbors Regressor, CatBoost Regressor, XGBoost Regressor provides the lowest test error at each horizon? (2) Does a model improve on persistence and seasonal-naive baselines? (3) How does error differ across the assigned sites? (4) How does estimator fit time change with sample size?

Forecast definition

Horizon	Target label	30-minute intervals ahead	Interpretation
24 hours	nitrate at t + 24 h	48	Nitrate mean at the same clock time one day later
7 days	nitrate at t + 7 d	336	Nitrate mean at the same clock time one week later

2. Dataset Description and Provenance

The original archive supplied for this analysis contains monthly NEON `NSW_15_minute` CSVs for product DP1.20033.001. The official NEON page describes this product as in-situ surface-water nitrate concentration, summarized as 15-minute means, with values reported in micromoles per liter. The original source files for the three assigned sites are included in `dataset/raw/`; `dataset/metadata/variables.csv`, `readme.txt` and `source_manifest.csv` support traceability.

Target: `surfWaterNitrateMean`. The modelling table retains future target values only when a valid 30-minute aggregated observation exists exactly at t+24 hours or t+7 days; labels are not interpolated.

Site coverage after quality filtering and 30-minute aggregation

Site	Input CSVs	Valid 15-min rows	30-min intervals	Mean NO3 (µmol/L)	Grid gaps %
LECO	92	177,852	89,271	13.33	19.9
LEWI	89	209,483	107,603	180.51	17.2
MCDI	89	192,639	97,735	24.86	24.3

The “grid gaps” column estimates the proportion of 30-minute timestamps missing between each site’s first and last retained interval. Gaps remain gaps: the pipeline does not fill missing target intervals with synthetic values.

3. Preprocessing and Feature Engineering

Preprocessing is performed from the raw source CSVs in the PySpark notebook so that the transformation is reproducible rather than dependent on a previously prepared model table.

Stage	Operation
1. Read source tables	Read only files matching `*NSW_15_minute*.csv` under the three assigned site folders and extract siteID from the source path.
2. Parse and type columns	Parse start/end timestamps as timestamps; cast nitrate means, summary measurements, uncertainty, standard error and finalQF to numeric types.
3. Apply QC rule	Keep valid timestamps, non-null nonnegative `surfWaterNitrateMean`, and `finalQF == 0`. NEON defines finalQF=0 as pass and finalQF=1 as fail.
4. Remove duplicates	Remove duplicate `(siteID, startDateTime)` records deterministically after filtering the quality flag.
5. Aggregate	Group adjacent 15-minute observations into non-overlapping 30-minute windows. Store each window end as the forecast origin. Keep a count of accepted 15-minute rows in each bin.
6. Construct targets	Join the time series to itself using exact timestamps to create future labels at +24 hours and +7 days. Missing future times do not form training labels.
7. Split chronologically	Within each site, use the first 80% of labelled times for training and later times for testing. Training rows are further restricted so their target timestamps fall before the cutoff.

Predictor dictionary

Feature	Role	Description
nitrate_mean	Current-value predictor	Mean of accepted source nitrate means in the completed current 30-minute bin.
lag_30m / lag_1h	Short-term history	Exact previous 30-minute and 1-hour nitrate means.
lag_24h / lag_7d	Seasonal history	Exact same-time value from one day or one week earlier, when available.
nitrate_min / nitrate_max	Interval summaries	Minimum and maximum source summaries inside the 30-minute bin.
uncertainty_mean / std_error_mean	Measurement context	Mean reported expanded uncertainty and standard error for the source observations.
valid_15min_count	Completeness feature	Number of accepted source observations in the 30-minute bin (typically 1 or 2).
hour_sin/cos, doy_sin/cos, month	Calendar features	Cyclic hour-of-day and day-of-year encodings plus month.

A 30-minute bin with one accepted 15-minute input is retained and explicitly marked by `valid_15min_count`. This avoids inventing values while preserving the fact that some bins are partial. A stricter complete-bin-only sensitivity run is recommended for the final scientific analysis.

4. Models and Evaluation Methodology

PySpark performs raw-data ingestion, QC, 30-minute aggregation, feature construction and chronological splitting. The three assigned Python estimators below run on a stratified sample per site, enabling diverse algorithms not all available in Spark MLlib. Their timings are driver-side estimator timings, not distributed MLlib fit times. The evaluation is run separately for the 24-hour and 7-day target labels.

Model	Type	Configuration / purpose
K-Nearest Neighbors Regressor	Instance-based ML	Predicts from nearby training examples after feature scaling.
CatBoost Regressor	Gradient-boosted decision trees	CatBoost builds sequential trees with ordered boosting and regularization.
XGBoost Regressor	Gradient-boosted decision trees	XGBoost uses regularized gradient-boosted trees with histogram splits.
Persistence	Naive baseline	Predicts the future value using the current completed 30-minute nitrate mean.
Seasonal Naive	Naive baseline	Uses the same-time nitrate mean from 24 hours or 7 days earlier.

Metrics

Metric	Interpretation
MAE	Average absolute prediction error; lower is better; expressed in µmol/L.
RMSE	Square root of mean squared error; penalizes larger misses; lower is better.
R^2	Variance-explained statistic relative to the test mean. Compare with site-specific scores because nitrate scales differ by site.
Fit / prediction seconds	Wall-clock time measured around model fitting and prediction actions on the execution machine.

Leakage prevention and comparison

The target is computed with exact-time self-joins, and no target label is included among model predictors. A chronological site-wise 80/20 split is used instead of a random split; a training row is eligible only if its future label still falls before the site's cutoff. The same prediction origins are used for the three ML models at a given horizon. Site-specific MAE/RMSE are also reported because pooled metrics can hide poor performance at one site when average concentrations differ substantially.

Reproducibility note: the notebook writes person-specific model metrics, per-site metrics, and estimator sample-size scaling under `outputs/tables/`. Fit/predict times for these Python estimators are not Spark MLlib training times.

5. Comparative Results - Local Reference Run

These are measured local reference results for the three algorithms assigned to this person. The reference uses up to 1,500 training rows per site and up to 2,000 future holdout rows per site. PySpark creates the dataset and chronological split; these local Python estimator timings are not Spark MLlib timings. Treat the table as a pilot comparison; the notebook reruns models and timings on the submission machine.

24-hour forecast results

Model	Test rows	MAE	RMSE	R^2	Fit (s)
K-Nearest Neighbors Regressor	6,000	7.922	12.901	0.972	0.02
CatBoost Regressor	6,000	5.986	11.228	0.979	0.53
XGBoost Regressor	6,000	5.989	11.425	0.978	0.35
Persistence	6,000	6.399	13.550	0.969	0.00
Seasonal naive	5,853	7.542	14.790	0.963	0.00

Lowest pooled ML RMSE at 24h: CatBoost Regressor (RMSE 11.228 µmol/L; MAE 5.986 µmol/L). Baselines remain important and should be judged separately at each horizon.

7-day forecast results

Model	Test rows	MAE	RMSE	R^2	Fit (s)
K-Nearest Neighbors Regressor	6,000	9.806	15.636	0.959	0.02
CatBoost Regressor	6,000	8.813	14.709	0.963	0.54
XGBoost Regressor	6,000	9.450	15.383	0.960	0.31
Persistence	6,000	9.698	17.524	0.948	0.00
Seasonal naive	5,661	11.527	19.730	0.935	0.00

Lowest pooled ML RMSE at 7d: CatBoost Regressor (RMSE 14.709 µmol/L; MAE 8.813 µmol/L). Baselines remain important and should be judged separately at each horizon.

Interpretation

Longer forecast horizons are generally harder, and the best model can vary by horizon and site. The baseline comparison matters: an ML method should only be claimed as an improvement where it beats persistence or seasonal-naive on the same test observations. This reference evaluation uses a stratified sample, so treat the metrics as pilot evidence rather than a definitive ranking on every eligible row.

Pooled R^2 can be artificially high where sites have very different mean concentrations. For a defensible scientific conclusion, prioritize the per-site metrics table and include a complete-bin-only sensitivity check before final submission.

6. Site-Specific Results and Runtime Analysis

Best model by site and horizon (local reference run; chosen by lowest site-specific RMSE among the three ML models)

Site	Mean NO3	24h best ML	24h RMSE	7d best ML	7d RMSE
LECO	13.3	CatBoost Regressor	3.81	K-Nearest Neighbors Regressor	4.96
LEWI	180.5	CatBoost Regressor	18.37	CatBoost Regressor	21.83
MCDI	24.9	CatBoost Regressor	5.11	CatBoost Regressor	12.11

Site-level errors may differ substantially because each assigned site has its own concentration scale, missing-data pattern and temporal behaviour. Interpret absolute errors with each site's distribution in view rather than assuming pooled metrics tell the whole story.

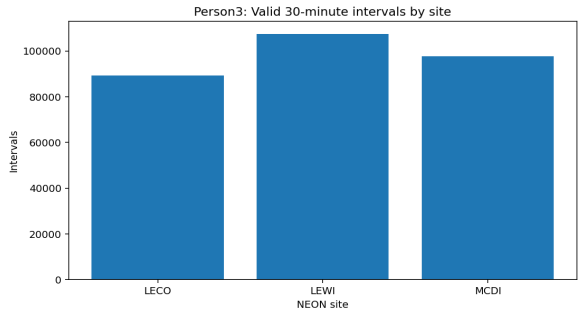
Reference execution-time / sample-size scaling

Fit rows	Fraction of sample	Fit seconds	Predict seconds	Scored rows
1,125	25%	0.235	0.020	3,000
2,250	50%	0.247	0.020	3,000
4,500	100%	0.305	0.022	3,000

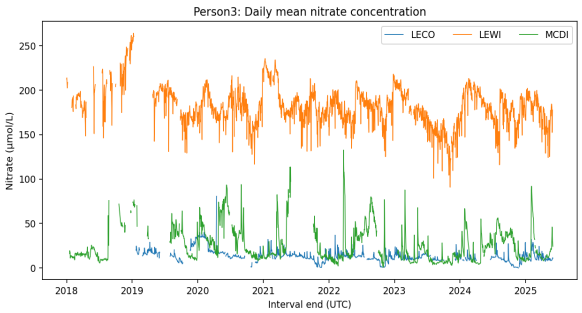
The reference timing uses a bounded local estimator sample. The notebook measures XGBoost Regressor at 25%, 50% and 100% of sampled training rows. These are machine-specific wall-clock seconds; for Person 2 and Person 3 they are driver-side Python estimator times, not Spark MLlib training times.

7. Visualisations

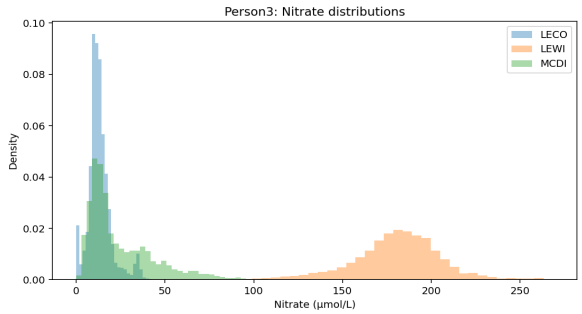
These figures summarize the included sites' time coverage, nitrate distribution and reference model/runtime comparisons. The notebook regenerates preprocessing plots and writes additional Spark comparison plots during execution.



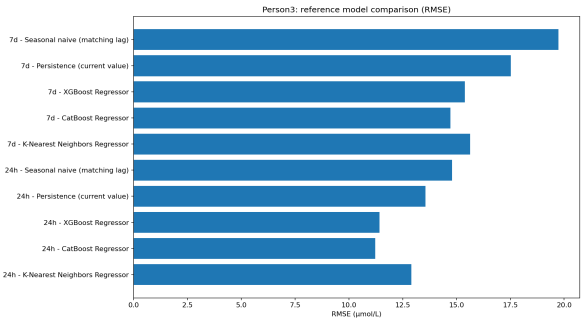
Valid 30-minute intervals



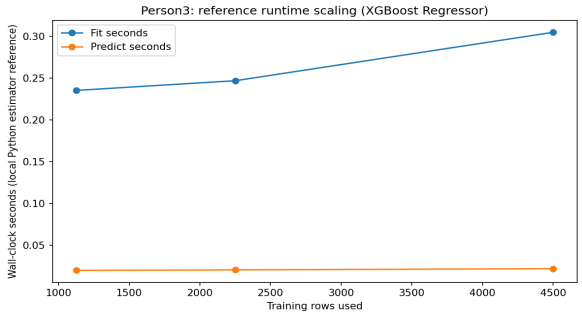
Daily mean nitrate time series



Nitrate distributions



Reference RMSE comparison



Reference runtime scaling

8. Discussion, Limitations and Conclusion

The pilot analysis shows that future nitrate means can be modelled from historical nitrate levels and calendar/interval features. The best estimator can vary by horizon and assigned site group, so rankings should be based on held-out error rather than assumptions about algorithm superiority. Persistence and seasonal-naive baselines provide a necessary check that a more complex model adds value.

Limitations and next steps

1. The reference model run uses a stratified capped sample (up to 1,500 training rows per site and 2,000 held-out rows per site) so the comparison is reproducible on ordinary student hardware. The notebook re-runs the models and measures timings in the actual execution environment.

2. The selected sites have different nitrate scales and missing-data patterns. Use site-level MAE/RMSE, normalized errors, and temporal plots rather than relying on pooled R^2 alone.

3. A 30-minute bin can contain one or two accepted 15-minute values. Report the `valid_15min_count` distribution and compare results after restricting to complete 2-observation bins.

4. Forecast quality depends on measurements being available at the exact future timestamp. Gaps are not interpolated; this avoids fabricated targets but reduces the number of train/test examples.

5. The archive contains the RELEASE-2026 versioned files. NEON published an algorithm/quality-flag update in June 2026; the official product page describes its scope. Verify the release and issue log before a final publication.

Conclusion

For Person3, the supplied workflow provides a reproducible structure for 24-hour and 7-day nitrate forecasting across LECO, LEWI, MCDI. It includes raw site-specific source data, preprocessing, three distinct assigned models (K-Nearest Neighbors Regressor, CatBoost Regressor, XGBoost Regressor), two naive baselines, MAE/RMSE/R^2 comparison, per-site scoring, plots, and a sample-size runtime experiment. The final submission should run the notebook, check generated metrics, and discuss how performance varies by site.

References

• NEON. Nitrate in surface water, DP1.20033.001. https://data.neonscience.org/data-products/DP1.20033.001

• NEON. Quick Start Guide for Nitrate in Surface Water, DP1.20033.001. https://data.neonscience.org/api/v0/documents/quick-start-guides/NEON.QSG.DP1.20033.001v1?fallback=html&inline=true

• NEON. Nitrate in surface water algorithm updates, June 12, 2026. https://www.neonscience.org/impact/observatory-blog/nitrate-surface-water-dp120033001-algorithm-updates

• Apache Spark. MLlib / DataFrame-based machine learning guide. https://spark.apache.org/docs/latest/ml-guide.html

• scikit-learn. Supervised learning reference and estimator API. https://scikit-learn.org/stable/supervised_learning.html

• CatBoost documentation. https://catboost.ai/en/docs/

• XGBoost documentation. https://xgboost.readthedocs.io/en/stable/

• pandas documentation. https://pandas.pydata.org/docs/

• Matplotlib documentation. https://matplotlib.org/stable/